

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №2
Реализация микросервисов

Выполнил:
Христофоров Владислав

Группа
К3340

Поток
БР 1.2

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Опираясь на документ, сформированный в рамках ДЗ-4 (Технический дизайн), реализовать разделение монолитного бэкенд-приложения на микросервисы. Обеспечить изолированность баз данных, настроить внутреннее межсервисное взаимодействие посредством синхронных HTTP-запросов и организовать единую точку входа в приложение.

Ход работы

1 Инфраструктурная изоляция (Изоляция СУБД)

В соответствии с паттерном *Database-per-Service*, единая база данных монолита была разделена на 4 независимые БД: `identity_db`, `recipes_db`, `social_db` и `media_db`. Для их развертывания в среде разработки написан новый конфигурационный файл `docker-compose.yml`, запускающий изолированные контейнеры PostgreSQL с пробросом портов (5432-5435) на локальную машину.

2 Рефакторинг моделей (TypeORM) и разрыв связей

Кодовая база была реорганизована в монорепозиторий (директория `services`), содержащий 5 независимых Node.js проектов. Для обеспечения слабой связности (Loose Coupling) в моделях баз данных (Entities) были физически удалены все междоменные связи (декораторы `@ManyToOne` и `@OneToMany`). Они заменены на обычные колонки строкового типа UUID (например, `author_id`, `user_id`). Это позволило микросервисам оперировать данными независимо друг от друга.

3 Реализация единой точки входа (API Gateway)

Разработан проксирующий шлюз на базе фреймворка Express.js и библиотеки `http-proxy-middleware`. В шлюз перенесена зона ответственности за аутентификацию. При поступлении внешнего запроса:

- 1) шлюз извлекает JWT из заголовка и делает синхронный запрос валидации к Identity Service,
- 2) в случае успеха шлюз пробрасывает извлеченные ID и Role пользователя в заголовки X-User-Id и X-User-Role,

3) запрос прозрачно перенаправляется (роутится) к целевому микросервису. В Gateway также интегрирована единая спецификация Swagger UI, агрегирующая документацию всех микросервисов.

4 Межсервисное взаимодействие и отказоустойчивость

Для компенсации разрыва внешних ключей в базах данных (Foreign Keys), реализована система синхронных REST-запросов между микросервисами с использованием библиотеки *axios*:

- валидация целостности: при попытке поставить лайк или оставить комментарий, Social Service совершает HTTP-запрос к Recipe Service (GET /internal/recipes/{id}/exists), чтобы убедиться в существовании целевого рецепта,
- агрегация профилей (Решение проблемы N+1): при формировании ленты постов блога, Social Service собирает массив уникальных *author_id* и отправляет единый Bulk-запрос в Identity Service для извлечения имен и аватарок авторов,
- распределенные транзакции (Оркестрация): при блокировке (или удалении) профиля, Identity Service выступает в роли оркестратора и рассылает запросы на каскадное применение *Soft Delete* к контенту пользователя в Recipe Service и Social Service.

Вывод

В ходе выполнения лабораторной работы монолитное бэкенд-приложение было успешно переведено на микросервисную архитектуру согласно ранее спроектированному техническому дизайну. Настроены 4 независимых доменных сервиса со своими базами данных и 1 инфраструктурный шлюз. Обеспечена целостность данных в распределенной среде посредством имплементации внутренних (internal) HTTP-эндпоинтов. Приложение полностью работоспособно, масштабируемо и готово к дальнейшей интеграции с брокерами сообщений для перевода части вызовов на асинхронную модель обмена событиями.